

Nama: [Mohammad Raifandi Baiqi]
NIM: [25120500002]
Kelas: [Comp2]
Case: Product / Inventory
API Repository: [<https://github.com/mohammadraifbaiqi-cyber/uts-comp2-1>]

Ringkasan Case

Tujuan API/Protocol

Pada praktikum ini dilakukan pengujian terhadap Product Inventory API yang tersedia pada Communication Protocol Demo App. Tujuan pengujian adalah untuk memahami bagaimana komunikasi antara client dan server dilakukan menggunakan protokol HTTP serta bagaimana data dipertukarkan menggunakan format JSON.

Melalui pengujian ini dilakukan beberapa operasi seperti melihat daftar produk, melihat detail produk, menambahkan produk baru, dan memperbaiki data produk. Selain itu, dilakukan analisis terhadap request, response, status code, header, dan struktur data yang digunakan oleh API.

Skenario Bisnis/Data

Product Inventory API mensimulasikan sistem manajemen inventori yang digunakan oleh perusahaan atau toko untuk mengelola data produk. Sistem ini menyimpan informasi produk seperti nama produk, kategori, harga, dan jumlah stok.

Pada skenario ini, pengguna dapat mengambil informasi seluruh produk yang tersedia, melihat detail produk tertentu berdasarkan ID, menambahkan produk baru ke dalam inventori, serta memperbaiki informasi produk yang sudah ada. Data yang dipertukarkan antara client dan server menggunakan format JSON sehingga mudah dibaca dan diproses oleh aplikasi.

Desain Request

Pengujian dilakukan menggunakan Product Inventory API yang berjalan pada Communication Protocol Demo App dengan base URL:

`http://127.0.0.1:8088`

Request 1 - Get All Products

Komponen	Nilai
Endpoint	<code>/api/products</code>

Method	GET
Headers	Tidak diperlukan
Body	Tidak ada
Expected Response	Daftar seluruh produk dalam format JSON dengan status code 200 OK

Request 2 - Get Product Detail

Komponen	Nilai
Endpoint	<code>/api/products/5</code>
Method	GET
Headers	Tidak diperlukan
Body	Tidak ada
Expected Response	Detail produk dengan ID 5 dalam format JSON dengan status code 200 OK

Request 3 - Create Product (Smart Watch)

Komponen	Nilai
Endpoint	<code>/api/products</code>
Method	POST
Headers	<code>Content-Type: application/json</code>
Body	JSON Product Data
Expected Response	Produk baru berhasil dibuat dan server mengembalikan data produk yang ditambahkan

Body Request:

```
{
  "name": "Smart watch",
  "category": "watch",
  "price": 132000,
  "stock": 22
}
```

Request 4 - Create Product (Monitor)

Komponen	Nilai
Endpoint	<code>/api/products</code>
Method	POST
Headers	<code>Content-Type: application/json</code>
Body	JSON Product Data
Expected Response	Produk baru berhasil ditambahkan ke dalam inventory dan server mengembalikan response JSON

Body Request:

```
{
  "name": "monitor",
  "category": "tech",
  "price": 112000,
  "stock": 10
}
```

Request 5 - Update Product

Komponen	Nilai
Endpoint	<code>/api/products/5</code>
Method	PUT
Headers	<code>Content-Type: application/json</code>
Body	JSON Product Data
Expected Response	Data produk dengan ID 5 berhasil diperbarui dan server mengembalikan data terbaru

Body Request:

```
{
  "name": "Smart watch",
  "category": "watch",
}
```

```
"price": 132000,  
"stock": 20  
}
```

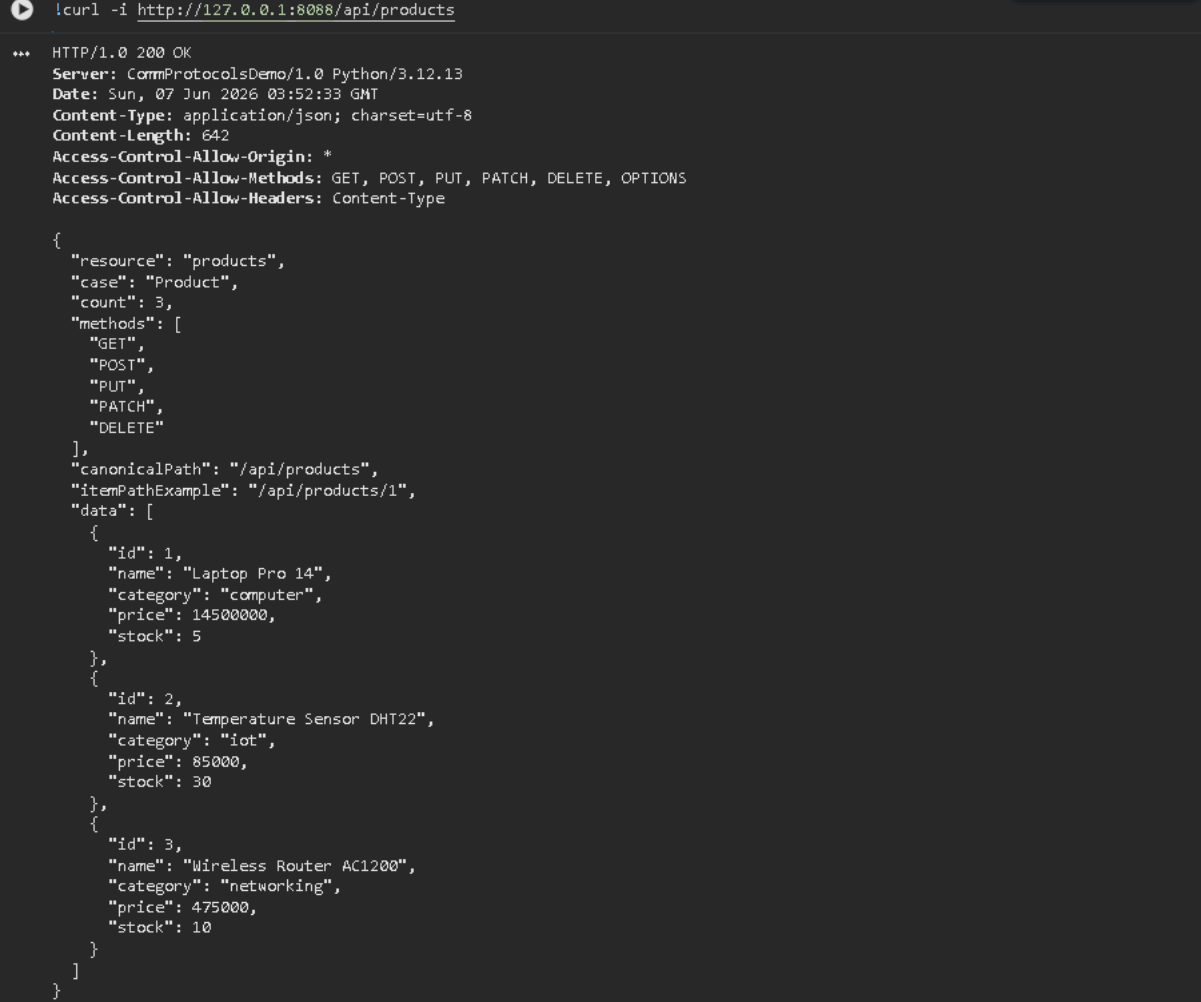
Evidence GET

GET Request 1 - Get All Products

Request

```
curl -i http://127.0.0.1:8088/api/products/
```

Screenshot



```
!curl -i http://127.0.0.1:8088/api/products  
... HTTP/1.0 200 OK  
Server: CommProtocolsDemo/1.0 Python/3.12.13  
Date: Sun, 07 Jun 2026 03:52:33 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 642  
Access-Control-Allow-Origin: *  
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS  
Access-Control-Allow-Headers: Content-Type  
  
{  
  "resource": "products",  
  "case": "Product",  
  "count": 3,  
  "methods": [  
    "GET",  
    "POST",  
    "PUT",  
    "PATCH",  
    "DELETE"  
  ],  
  "canonicalPath": "/api/products",  
  "itemPathExample": "/api/products/1",  
  "data": [  
    {  
      "id": 1,  
      "name": "Laptop Pro 14",  
      "category": "computer",  
      "price": 14500000,  
      "stock": 5  
    },  
    {  
      "id": 2,  
      "name": "Temperature Sensor DHT22",  
      "category": "iot",  
      "price": 85000,  
      "stock": 30  
    },  
    {  
      "id": 3,  
      "name": "Wireless Router AC1200",  
      "category": "networking",  
      "price": 475000,  
      "stock": 10  
    }  
  ]  
}
```

Penjelasan

Request GET digunakan untuk mengambil seluruh data produk yang tersedia pada sistem inventory. Server berhasil memproses request dan mengembalikan response dengan status

code **200 OK**. Data yang diterima menggunakan format JSON dan berisi informasi jumlah produk serta daftar produk yang tersimpan pada server.

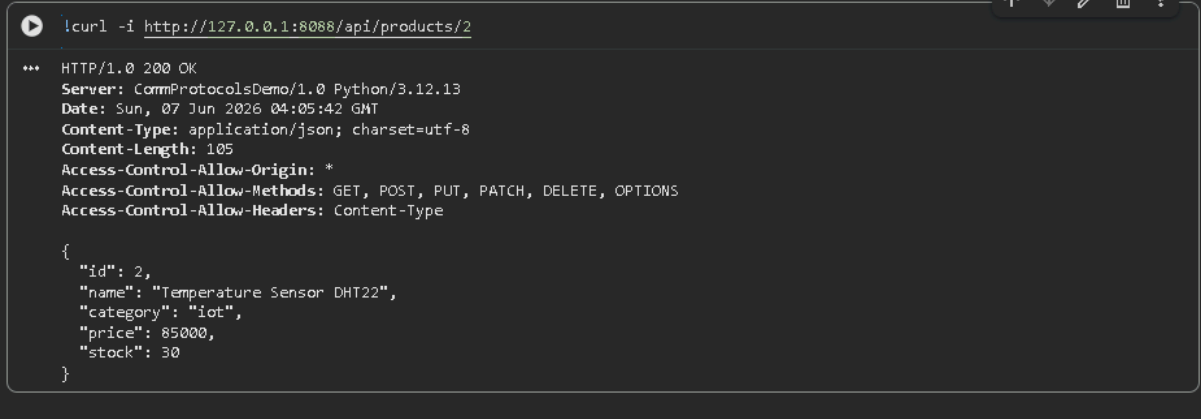
Response juga menunjukkan metadata seperti nama resource, jumlah data, dan array data produk. Setiap produk memiliki atribut id, name, category, price, dan stock.

GET Request 2 - Get Product Detail

Request

```
curl -i http://127.0.0.1:8088/api/products/5
```

Screenshot



```
!curl -i http://127.0.0.1:8088/api/products/2

*** HTTP/1.0 200 OK
Server: CommProtocolsDemo/1.0 Python/3.12.13
Date: Sun, 07 Jun 2026 04:05:42 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 105
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type

{
  "id": 2,
  "name": "Temperature Sensor DHT22",
  "category": "iot",
  "price": 85000,
  "stock": 30
}
```

Penjelasan

Request GET digunakan untuk mengambil detail produk berdasarkan ID. Pada pengujian ini digunakan ID produk 5 sehingga server mengembalikan informasi lengkap produk tersebut.

Response diterima dalam format JSON dengan status code **200 OK**. Data yang ditampilkan meliputi identitas produk, nama produk, kategori, harga, dan jumlah stok yang tersedia. Penggunaan endpoint dengan parameter ID memungkinkan client memperoleh data yang lebih spesifik dibandingkan endpoint daftar produk.

Contoh data yang diperoleh:

```
{
  "id": 5,
  "name": "Smart watch",
  "category": "watch",
  "price": 132000,
  "stock": 20
}
```

Dari hasil tersebut dapat diketahui bahwa API mampu menyediakan informasi detail produk secara individual melalui endpoint yang berbeda.

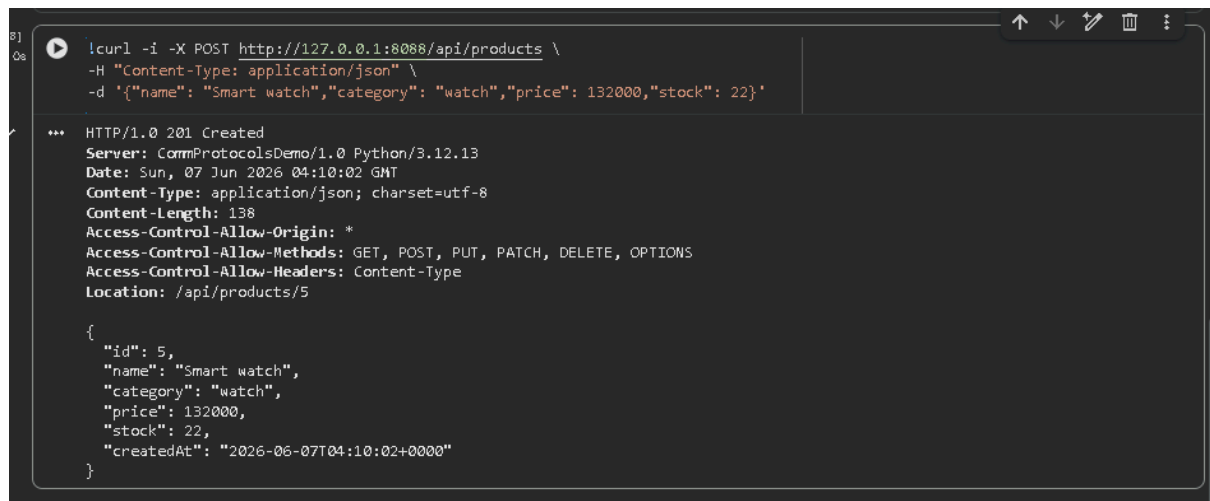
Evidence POST

POST Request 1 - Create Product (Smart Watch)

Request

```
curl -i -X POST http://127.0.0.1:8088/api/products \
-H "Content-Type: application/json" \
-d '{"name": "Smart watch", "category": "watch", "price": 132000, "stock": 22}'
```

Screenshot

A screenshot of a terminal window with a dark background. The terminal shows a curl command being executed to send a POST request to the endpoint http://127.0.0.1:8088/api/products. The request headers include 'Content-Type: application/json' and the body is a JSON object representing a smart watch. The response from the server is displayed, showing an HTTP 201 status code and a JSON response with product details including an ID, name, category, price, stock, and a timestamp.

```
!curl -i -X POST http://127.0.0.1:8088/api/products \
-H "Content-Type: application/json" \
-d '{"name": "Smart watch", "category": "watch", "price": 132000, "stock": 22}'

*** HTTP/1.0 201 Created
Server: CommProtocolsDemo/1.0 Python/3.12.13
Date: Sun, 07 Jun 2026 04:10:02 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 138
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type
Location: /api/products/5

{
  "id": 5,
  "name": "Smart watch",
  "category": "watch",
  "price": 132000,
  "stock": 22,
  "createdAt": "2026-06-07T04:10:02+0000"
}
```

Penjelasan

Request POST digunakan untuk menambahkan data produk baru ke dalam sistem inventory. Data produk dikirim dalam format JSON melalui request body yang berisi nama produk, kategori, harga, dan jumlah stok.

Server berhasil menerima request dan memproses data yang dikirim oleh client. Setelah proses berhasil, server mengembalikan response dalam format JSON yang berisi informasi produk yang telah ditambahkan. Penggunaan method POST sesuai dengan prinsip REST API karena digunakan untuk membuat resource baru pada server.

Data yang dikirim:

```
{
  "name": "Smart watch",
  "category": "watch",
  "price": 132000,
  "stock": 22
}
```

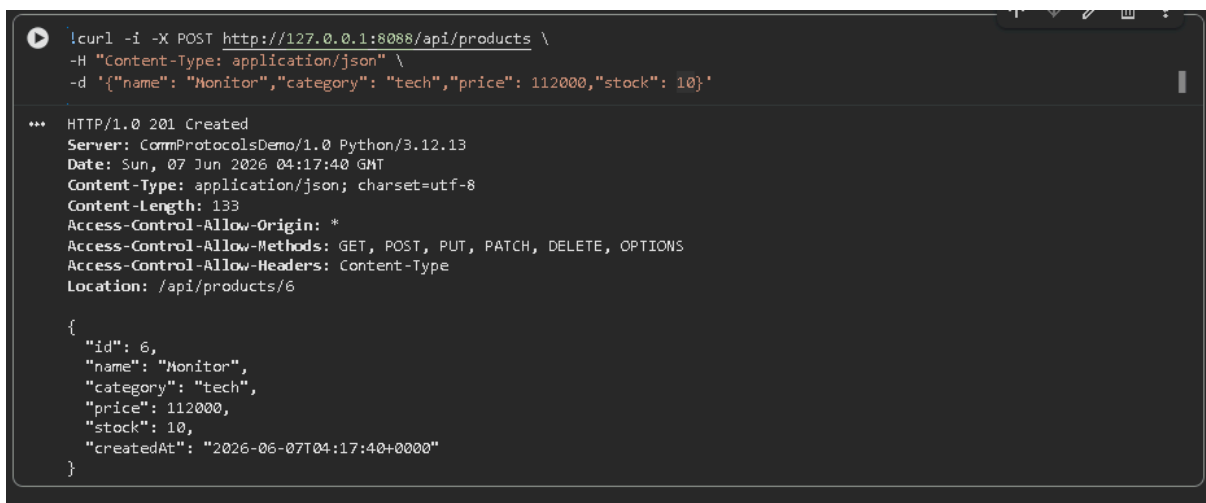
```
}
```

POST Request 2 - Create Product (Monitor)

Request

```
curl -i -X POST http://127.0.0.1:8088/api/products \
-H "Content-Type: application/json" \
-d '{"name": "monitor", "category": "tech", "price": 112000, "stock": 10}'
```

Screenshot

A screenshot of a terminal window with a dark background. The top part shows the command: `!curl -i -X POST http://127.0.0.1:8088/api/products \`, `-H "Content-Type: application/json" \`, and `-d '{"name": "Monitor", "category": "tech", "price": 112000, "stock": 10}'`. Below the command, the response is displayed, starting with `HTTP/1.0 201 Created`. The response headers include `Server: CommProtocolsDemo/1.0 Python/3.12.13`, `Date: Sun, 07 Jun 2026 04:17:40 GMT`, `Content-Type: application/json; charset=utf-8`, `Content-Length: 133`, `Access-Control-Allow-Origin: *`, `Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS`, `Access-Control-Allow-Headers: Content-Type`, and `Location: /api/products/6`. The response body is a JSON object: `{ "id": 6, "name": "Monitor", "category": "tech", "price": 112000, "stock": 10, "createdAt": "2026-06-07T04:17:40+0000" }`.

Penjelasan

Request POST kedua digunakan untuk menambahkan produk baru dengan kategori teknologi. Sama seperti pengujian sebelumnya, data produk dikirim menggunakan format JSON melalui request body.

Server menerima data dan menyimpannya sebagai resource baru pada sistem inventory. Response yang diberikan server menunjukkan bahwa proses penambahan data berhasil dilakukan. Dari hasil ini dapat diketahui bahwa endpoint `/api/products` mendukung operasi pembuatan data baru menggunakan method POST.

Data yang dikirim:

```
{
  "name": "monitor",
  "category": "tech",
  "price": 112000,
  "stock": 10
}
```

Hasil Pengamatan

Berdasarkan kedua pengujian POST, API mampu menerima data dalam format JSON dan memproses pembuatan resource baru pada sistem inventory. Setiap request menggunakan header:

Content-Type: application/json

yang menunjukkan bahwa payload dikirim menggunakan format JSON. Hasil pengujian membuktikan bahwa komunikasi antara client dan server berjalan dengan baik serta sesuai dengan prinsip REST API untuk operasi Create (POST).

Analisis Response

Analisis Status Code

Berdasarkan hasil pengujian API, server mengembalikan beberapa status code yang menunjukkan hasil pemrosesan request.

200 OK

Status code 200 OK menunjukkan bahwa request berhasil diproses oleh server dan data berhasil dikembalikan kepada client. Status ini ditemukan pada request GET untuk mengambil daftar produk dan detail produk.

Contoh:

HTTP/1.0 200 OK

400 Bad Request

Status code 400 Bad Request menunjukkan bahwa request yang dikirim tidak sesuai dengan aturan API. Error ini ditemukan ketika melakukan request POST ke endpoint yang tidak mendukung operasi tersebut.

Contoh:

HTTP/1.0 400 Bad Request

Status ini membantu client mengetahui bahwa request perlu diperbaiki sebelum dikirim kembali.

Analisis Header Response

Contoh header yang diterima dari server:

Content-Type: application/json; charset=utf-8

Content-Length: 90

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS

Access-Control-Allow-Headers: Content-Type

Penjelasan:

Content-Type

Menunjukkan bahwa data response dikirim dalam format JSON dengan encoding UTF-8 sehingga dapat diproses oleh berbagai platform dan bahasa pemrograman.

Content-Length

Menunjukkan ukuran data response yang dikirim oleh server dalam satuan byte.

Access-Control-Allow-Origin

Menunjukkan bahwa API mengizinkan akses dari berbagai origin sehingga mendukung komunikasi lintas domain (Cross-Origin Resource Sharing/CORS).

Access-Control-Allow-Methods

Menampilkan metode HTTP yang didukung oleh API yaitu GET, POST, PUT, PATCH, DELETE, dan OPTIONS.

Access-Control-Allow-Headers

Menentukan header yang diperbolehkan saat client mengirim request ke server.

Analisis Body Response

Body response dikirim menggunakan format JSON.

Contoh response GET:

```
{
  "resource": "products",
  "count": 3,
  "data": [
    {
      "id": 1,
      "name": "Laptop Pro 14",
      "category": "electronics",
      "price": 14500000,
      "stock": 12
    }
  ]
}
```

Struktur response terdiri dari:

- resource : nama resource yang diakses.
- count : jumlah data yang tersedia.
- data : kumpulan data produk dalam bentuk array.
- id : identitas unik produk.
- name : nama produk.
- category : kategori produk.
- price : harga produk.
- stock : jumlah stok produk.

Penggunaan JSON membuat data lebih ringan dan mudah dibaca dibandingkan format lain seperti XML.

Analisis Error Handling

API menyediakan mekanisme error handling yang cukup baik dengan memberikan status code dan pesan kesalahan yang jelas.

Contoh error response:

```
{  
  
  "error": "Use POST /api/products to create a new resource",  
  
  "resource": "products"  
}
```

Error tersebut muncul ketika client mengirim request POST ke endpoint yang tidak sesuai. Selain memberikan status code 400 Bad Request, server juga menjelaskan penyebab kesalahan sehingga client dapat memperbaiki request yang dikirim.

Mekanisme ini membantu meningkatkan keandalan API karena pengguna dapat mengetahui kesalahan yang terjadi tanpa harus menebak penyebab kegagalan request.

Analisis Encoding / Traffic

Format Encoding yang Digunakan

Pada pengujian Product Inventory API, seluruh komunikasi antara client dan server menggunakan format **JSON (JavaScript Object Notation)**.

Contoh response yang diterima dari server:

```
{  
  
  "resource": "products",  
  
  "count": 3,  
  
  "data": [  
  
    {  
  
      "id": 5,  
  
      "name": "Smart watch",  
  
      "category": "watch",  
  
      "price": 132000,  
  
      "stock": 20
```

```
}  
]  
}
```

Format JSON digunakan karena memiliki struktur yang sederhana, ringan, dan mudah diproses oleh berbagai bahasa pemrograman.

Struktur Data JSON

Data produk terdiri dari beberapa atribut:

Field	Tipe Data	Keterangan
id	Integer	Identitas produk
name	String	Nama produk
category	String	Kategori produk
price	Integer	Harga produk
stock	Integer	Jumlah stok

Selain itu terdapat atribut tambahan pada response daftar produk:

Field	Keterangan
resource	Nama resource yang diakses

count	Jumlah data yang tersedia
data	Kumpulan data produk

Analisis Traffic HTTP

Komunikasi dilakukan menggunakan protokol HTTP dengan pertukaran data berbentuk JSON.

Contoh request:

GET /api/products HTTP/1.1

Contoh response:

HTTP/1.0 200 OK

Content-Type: application/json; charset=utf-8

Dari header tersebut dapat diketahui bahwa server mengirimkan data menggunakan format JSON dengan encoding UTF-8.

Keunggulan JSON

Beberapa alasan penggunaan JSON pada API ini adalah:

- Ukuran data lebih kecil dibanding XML.
- Mudah dibaca manusia (human-readable).
- Mudah diproses oleh aplikasi client maupun server.
- Menjadi standar umum pada implementasi REST API modern.

Kesimpulan

Berdasarkan hasil pengujian, Product Inventory API menggunakan format JSON sebagai mekanisme encoding data antara client dan server. Seluruh request dan response berhasil diproses menggunakan protokol HTTP dengan payload JSON yang ringan, terstruktur, dan mudah dipahami.

Kesimpulan

Berdasarkan hasil pengujian Product Inventory API pada Communication Protocol Demo App, dapat disimpulkan bahwa komunikasi antara client dan server berhasil dilakukan menggunakan protokol HTTP dengan format data JSON. Pengujian GET berhasil digunakan untuk mengambil daftar produk dan detail produk, sedangkan pengujian POST berhasil digunakan untuk menambahkan data produk baru ke dalam sistem inventory. Selain itu, API juga menyediakan mekanisme validasi dan error handling yang ditunjukkan melalui status code serta pesan error yang informatif.

Temuan Utama

- API menggunakan arsitektur REST dengan method HTTP seperti GET, POST, dan PUT.
- Data dikirim dan diterima menggunakan format JSON.
- Setiap request menghasilkan response yang disertai status code dan header yang sesuai.
- API mampu menangani request yang valid maupun tidak valid melalui mekanisme error handling.

Kendala

- Pada tahap awal terdapat kendala saat menjalankan server di Google Colab karena beberapa port sudah digunakan oleh proses lain.
- Akses ke localhost Colab tidak dapat dilakukan secara langsung dari browser lokal sehingga perlu menggunakan port forwarding dari Colab.
- Beberapa endpoint memiliki aturan penggunaan method tertentu sehingga menghasilkan error ketika request tidak sesuai dengan dokumentasi API.

Pembelajaran Protocol

Melalui praktikum ini diperoleh pemahaman mengenai cara kerja komunikasi client-server menggunakan protokol HTTP. Selain itu, dipelajari penggunaan method GET untuk mengambil data dan POST untuk menambahkan data pada REST API. Praktikum ini juga memberikan pemahaman mengenai pentingnya status code HTTP, header response, format JSON, serta mekanisme error handling dalam membangun layanan web yang andal dan mudah digunakan oleh client.